

PROYECTO INTEGRADO DE FIN DE CICLO

Infraestructura de Alta Disponibilidad y Orquestación con Kubernetes sobre Proxmox



AUTOR: Joseph Borrego Panal

CURSO: 2º de Administración de Sistemas Informáticos en Red (ASIR)

AÑO: 2025 / 2026

Tabla de contenido

- 1. Introducción3
 - 1.1. Contexto del Proyecto3
 - 1.2. Justificación y Problemática.....3
 - 1.3. Objetivos del Proyecto4
 - 1.3.1. Objetivo General4
 - 1.3.2. Objetivos Específicos.....4
 - 1.4. Alcance y Metodología.....4
- 2. Análisis de necesidades.....5
 - 2.1. Identificación del Problema y Justificación.....5
 - 2.2. Análisis de Viabilidad5
 - 2.2.1. Viabilidad Técnica5
 - 2.2.2. Viabilidad Económica6
 - 2.2.3. Viabilidad Temporal6
 - 2.3. Requisitos del Sistema6
 - 2.3.1. Requisitos Funcionales.....6
 - 2.3.2. Requisitos No Funcionales7
 - 2.4. Stack Tecnológico Propuesto7
- 3. Diseño del Sistema7
 - 3.1. Arquitectura de Niveles7
 - 3.1.1. Capa de Virtualización (Hipervisor).....9
 - 3.1.2. Capa de Orquestación (*Kubernetes*)9
 - 3.2. Topología Lógica y Nodos9
 - 3.3. Diseño de Red y Tráfico.....10
 - 3.3.1. Red Interna (CNI).....10
 - 3.3.2. Balanceo de Carga y Acceso Externo10
 - 3.4. Estrategia de Almacenamiento Persistente.....10
 - 3.5. Automatización y Gestión (IaC).....11
- 4. Implementación11
 - 4.1. Fase 1: Virtualización y Preparación del Host (*Proxmox*)11
 - 4.1.1. Creación de la Plantilla Base (Gold Template)12
 - 4.2. Fase 2: Configuración de Red y Nodos.....12
 - 4.3. Fase 3: Preparación del Kernel y Runtime (*containerd*)13
 - 4.4. Fase 4: Inicialización y Red de Pods13

4.5. Fase 5: Servicios de Red (<i>MetalLB</i> e <i>Ingress</i>)	14
4.6. Fase 6: Almacenamiento Persistente NFS	15
4.7. Fase 7: Automatización con <i>Ansible</i> (<i>IaC</i>)	15
4.8. Fase 8: Despliegue de Aplicación Real (WordPress)	16
5. Resultados y pruebas	17
5.1. Despliegue de Servicios Productivos	17
5.1.1. Ecosistema WordPress (CMS + DB)	17
5.1.2. Juego Estático: "3 en Raya" (Alta Disponibilidad)	17
5.2. Prueba de Alta Disponibilidad (HA)	18
5.3. Prueba de Escalado Dinámico (Horizontal Pod Autoscaling)	18
5.4. Validación de Persistencia y Aprovisionamiento	18
6. Conclusiones	19
6.1. Cumplimiento de Objetivos	19
6.2. Valor Técnico de la Solución	19
6.3. Desafíos Superados y Aprendizajes	19
6.4. Líneas de Trabajo Futuras	20
6.5. Valoración Final	20
7. Anexos y Referencias	21
7.1 Webgrafía y Documentación Oficial	21
7.1.1. Virtualización y Sistema Base	21
7.1.2. Orquestación y Contenedores (Kubernetes)	21
7.1.3. Redes, Balanceo de Carga y Almacenamiento	21
7.1.4. Automatización y Gestión	22

1. Introducción

1.1. Contexto del Proyecto

En el escenario tecnológico actual, la administración de sistemas informáticos ha experimentado una transformación radical. Hemos pasado de gestionar servidores físicos individuales con aplicaciones monolíticas a orquestar ecosistemas complejos basados en microservicios y contenedores. Este cambio no es caprichoso; responde a la necesidad de las empresas de obtener una mayor agilidad, escalabilidad y, sobre todo, disponibilidad.

Este proyecto integrado nace en el marco del ciclo formativo de grado superior en Administración de Sistemas Informáticos en Red (ASIR). Se centra en la creación de una infraestructura *on-premise* profesional que utiliza la virtualización de alto rendimiento con *Proxmox VE* para sostener un clúster de *Kubernetes* (K8s). Esta combinación permite aprovechar el hardware local para ofrecer servicios con un nivel de resiliencia y automatización que, tradicionalmente, solo estaba al alcance de grandes centros de datos o proveedores de nube pública.

1.2. Justificación y Problemática

La motivación técnica de este trabajo surge de la identificación de problemas reales en las arquitecturas IT convencionales:

1. **Puntos únicos de fallo (SPOF):** En sistemas tradicionales, la caída de un servidor físico implica la interrupción inmediata del servicio.
2. **Infrautilización de recursos:** El despliegue de aplicaciones directamente sobre el hardware a menudo desperdicia capacidad de cómputo.
3. **Escalado complejo:** Aumentar la capacidad de una aplicación suele requerir intervenciones manuales tediosas y propensas a errores.

Para resolver esto, se propone una solución basada en la **orquestación de contenedores**. *Kubernetes* permite que, si un nodo falla, las aplicaciones se reprogramen automáticamente en otros nodos sanos, garantizando la continuidad del negocio. Además, la implementación de la **Infraestructura como Código (IaC)** mediante Ansible asegura que el crecimiento del sistema sea rápido, consistente y libre de errores humanos.

1.3. Objetivos del Proyecto

1.3.1. Objetivo General

Diseñar, implementar y documentar una infraestructura virtualizada de alta disponibilidad capaz de orquestar servicios contenerizados mediante *Kubernetes*, garantizando la persistencia de datos y el balanceo de carga automático en una red local.

1.3.2. Objetivos Específicos

Para alcanzar el objetivo general, se han definido los siguientes hitos técnicos:

- **Virtualización Eficiente:** Configurar *Proxmox VE* como hipervisor base para gestionar de forma flexible los recursos físicos.
- **Despliegue de Orquestación:** Inicializar un clúster de *Kubernetes v1.30* optimizado, utilizando *containerd* como *runtime* de bajo nivel.
- **Optimización de Red:** Resolver conflictos críticos de red en entornos bare-metal, como la desactivación de IPv6 y la gestión de *Cgroups* mediante *systemd*.
- **Balanceo de Carga y Enrutamiento:** Implementar *MetalLB* para la gestión de IPs externas en Capa 2 y *Nginx Ingress Controller* para el enrutamiento de servicios mediante nombres de dominio (L7).
- **Persistencia Descentralizada:** Configurar un servidor *NFS* dedicado que proporcione almacenamiento persistente dinámico, permitiendo que los datos sobrevivan a la destrucción de los contenedores.
- **Automatización Avanzada:** Desarrollar *Playbooks* de Ansible para el aprovisionamiento desatendido de nuevos nodos, logrando un escalado horizontal real.

1.4. Alcance y Metodología

El alcance del proyecto abarca desde el montaje del hardware y la configuración de la *BIOS* (*activación de SVM/VT-x*) hasta el despliegue de aplicaciones reales como *WordPress* o juegos estáticos (*3 en Raya*), pasando por toda la capa de red y almacenamiento.

La metodología de trabajo empleada ha sido **Kanban**, gestionada a través de *GitHub Projects*. Este enfoque ha permitido una visualización clara del flujo de trabajo, dividiendo la implementación

en fases secuenciales que garantizan que cada componente (*red, almacenamiento, orquestación*) sea estable antes de avanzar al siguiente nivel.

2. Análisis de necesidades

2.1. Identificación del Problema y Justificación

El proyecto se sitúa en un entorno corporativo ficticio bajo la entidad "*DevOps Solutions S.L.*", la cual enfrenta los retos típicos de las infraestructuras tradicionales basadas en servidores independientes. Los principales problemas detectados que justifican esta implementación son:

- **Tiempos de inactividad críticos:** Cualquier fallo en el hardware físico detiene los servicios, provocando pérdidas de disponibilidad.
- **Escalado vertical ineficiente:** Para aumentar la potencia de un servicio, se requiere ampliar el hardware de la máquina específica, lo cual es costoso y requiere paradas técnicas.
- **Inconsistencia de entornos:** Las diferencias en las configuraciones manuales entre servidores generan errores difíciles de depurar en producción.

La solución propuesta es la migración a una arquitectura de microservicios orquestada por **Kubernetes**. Se elige esta tecnología por ser el estándar de facto en la industria, ofreciendo portabilidad y una gestión declarativa de los recursos.

2.2. Análisis de Viabilidad

2.2.1. Viabilidad Técnica

La viabilidad técnica ha quedado demostrada tras la resolución exitosa de la fase de inicialización del clúster. El hardware utilizado permite ejecutar el stack propuesto con un rendimiento nativo. Se ha validado la compatibilidad de **containerd** como motor de ejecución (*runtime*) bajo la jerarquía de **Cgroups** de sistema en *Ubuntu 24.04*. Asimismo, la resolución de conflictos específicos de red (*Netplan* y *DNS*) asegura la estabilidad del plugin *CNI Flannel*.

2.2.2. Viabilidad Económica

El coste de adquisición de software para este proyecto es de **0€**. Se ha apostado íntegramente por soluciones Open Source con licencias permisivas (*GPL, Apache 2.0, MIT*):

- **Proxmox VE:** Alternativa gratuita y flexible a soluciones propietarias como *VMware ESXi*.
- **Kubernetes y Docker/Containerd:** Estándares abiertos sin costes de licenciamiento por nodo.
- **Ansible:** Herramienta de automatización sin agentes que reduce costes operativos.

2.2.3. Viabilidad Temporal

El proyecto se ha estructurado en **9 fases** secuenciales. La resolución temprana de los problemas de red y kernel en las primeras etapas ha reducido significativamente el riesgo de retrasos en el despliegue de los servicios superiores como *MetalLB, Ingress* y persistencia NFS.

2.3. Requisitos del Sistema

2.3.1. Requisitos Funcionales

El sistema debe cumplir con las siguientes capacidades operativas:

- **Despliegue Declarativo:** Permitir la gestión de aplicaciones mediante ficheros de configuración *YAML*.
- **Balanceo de Carga L2:** Asignar automáticamente IPs de la red local a los servicios mediante *MetalLB*.
- **Enrutamiento L7:** Redirigir tráfico *HTTP* basado en nombres de dominio a través de un *Ingress Controller*.
- **Auto-recuperación (Self-healing):** Recuperar automáticamente un contenedor si este falla o el nodo donde reside se detiene.
- **Persistencia de Datos:** Garantizar que la información de las aplicaciones sea independiente del nodo físico mediante almacenamiento externo.
- **Automatización IaC:** Capacidad de configurar nuevos nodos desde cero de forma desatendida mediante Ansible.

2.3.2. Requisitos No Funcionales

Para asegurar la calidad del entorno profesional, se establecen los siguientes parámetros:

- **Alta Disponibilidad:** Los servicios web deben seguir operativos ante la caída de un nodo *Worker*.
- **Escalabilidad Horizontal:** Posibilidad de añadir nuevos trabajadores al clúster sin interrupción del servicio.
- **Consistencia de Red:** Exclusión estricta de tráfico IPv6 para evitar latencias y errores de direccionamiento interno.
- **Rendimiento:** Uso de *systemd* como driver de *Cgroup* para una integración profunda con el kernel de *Ubuntu 24.04*.

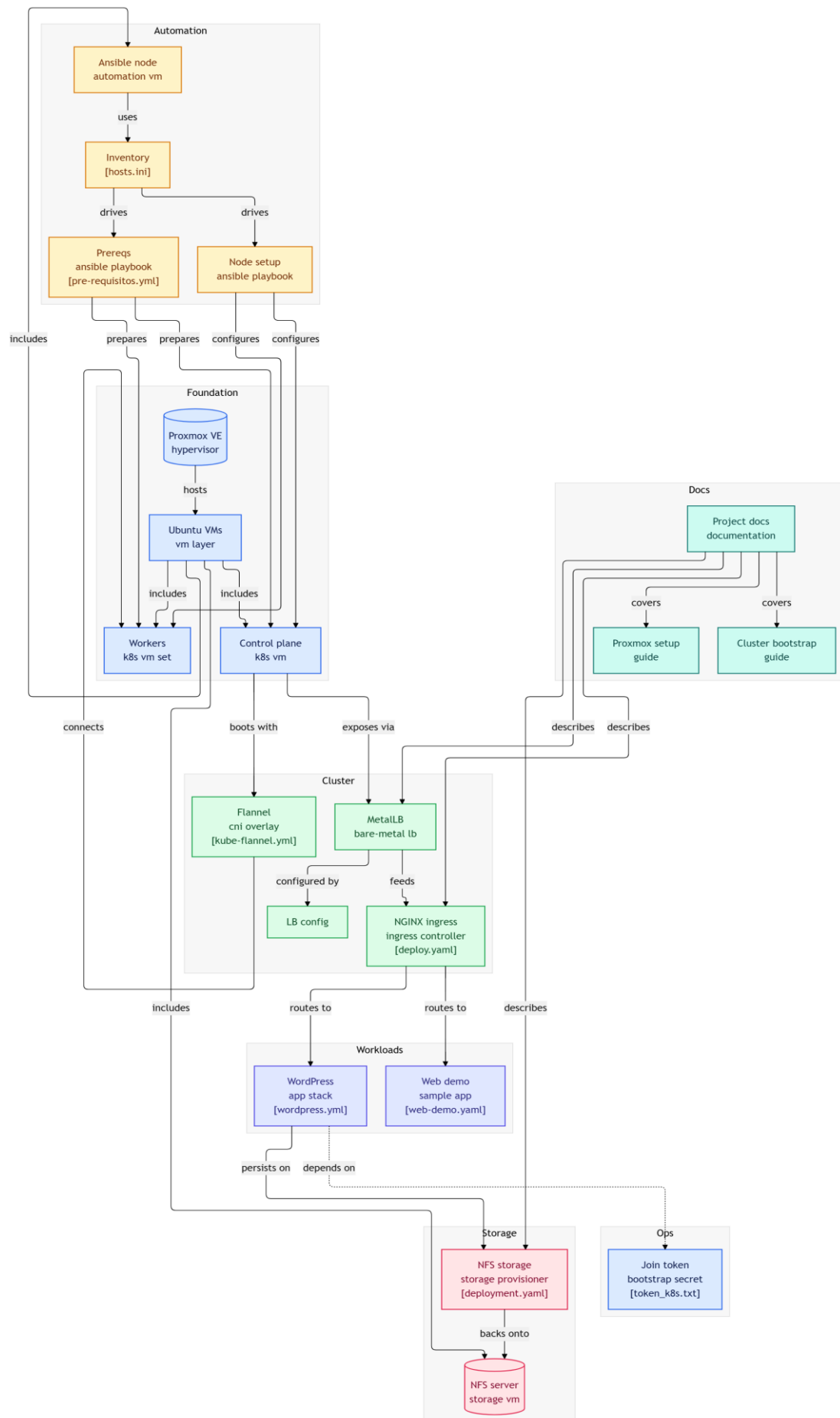
2.4. Stack Tecnológico Propuesto

Categoría	Tecnología / Herramienta
Hipervisor	Proxmox VE 9.x
Sistema Operativo	Ubuntu Server 24.04 LTS
Orquestación y Runtime	Kubernetes v1.30, Containerd
Red y Balanceo	Flannel (CNI), MetalLB, Nginx Ingress
Almacenamiento	Servidor NFS (Network File System)
Automatización	Ansible (Infraestructura como Código)

3. Diseño del Sistema

3.1. Arquitectura de Niveles

La arquitectura de este proyecto se ha diseñado bajo un enfoque modular de microservicios y alta disponibilidad. El sistema se organiza en cuatro niveles lógicos que interactúan entre sí, garantizando que el fallo en una capa superior no comprometa la integridad de los datos en las capas inferiores.



**Arquitectura modular por niveles*

3.1.1. Capa de Virtualización (Hypervisor)

La base de la infraestructura es un servidor físico que ejecuta **Proxmox VE**. Se ha seleccionado esta plataforma frente a alternativas propietarias por su flexibilidad para gestionar máquinas virtuales (*KVM*) y su base sólida en *Debian Linux*.

- **Configuración de Hardware:** Activación de **SVM Mode** en la BIOS para permitir que el procesador preste sus capacidades de virtualización directamente a las máquinas virtuales.
- **Gestión de Recursos:** *Proxmox* permite la asignación dinámica de CPU y memoria, además de la creación de plantillas (*templates*) para asegurar que todos los nodos del clúster partan de una configuración idéntica.

3.1.2. Capa de Orquestación (Kubernetes)

Sobre el hipervisor se despliega un clúster de **Kubernetes v1.30**. El diseño contempla la separación de roles entre el plano de control (*Control Plane*) y el plano de datos (*Worker Nodes*).

3.2. Topología Lógica y Nodos

Se ha diseñado una topología de red estática para evitar interrupciones por renovaciones de concesiones DHCP. Todas las máquinas virtuales utilizan **Ubuntu Server 24.04 LTS** como sistema operativo base.

Nodo	Función	Recursos (vCPU/RAM)	IP Estática
k8s-master	Cerebro del clúster (API, etcd, Scheduler)	2 vCPUs / 2 GB	192.168.1.110
k8s-worker-01	Ejecución de Pods y servicios	2 vCPUs / 2 GB	192.168.1.111
k8s-worker-02	Redundancia de cargas de trabajo	2 vCPUs / 2 GB	192.168.1.112
ansible-server	Gestión de Infraestructura como Código (IaC)	1 vCPU / 1 GB	192.168.1.115
nfs-server	Almacenamiento persistente centralizado	1 vCPU / 1 GB	192.168.1.116

***Nota sobre Alta Disponibilidad (HA):** El diseño actual centra la HA en los nodos Worker. Debido a limitaciones de hardware, se utiliza un único nodo *Master*, aunque se reconoce que en entornos de producción crítica se requeriría un *Control Plane* redundante para eliminar el punto único de fallo en la gestión.

3.3. Diseño de Red y Tráfico

3.3.1. Red Interna (CNI)

La comunicación entre los pods se realiza mediante el plugin **Flannel**, utilizando una red de *overlay* en el rango **10.244.0.0/16**. Esto permite que cada contenedor tenga su propia IP interna, independientemente del nodo en el que resida.

3.3.2. Balanceo de Carga y Acceso Externo

El diseño soluciona la carencia de balanceadores nativos en entornos *bare-metal* mediante dos componentes clave:

- **MetalLB (Capa 2):** Implementado con *strictARP: true* para anunciar y asignar IPs reales de la red local (rango **192.168.1.200 - 192.168.1.250**) a los servicios del clúster.
- **Nginx Ingress Controller (Capa 7):** Actúa como proxy inverso, permitiendo el enrutamiento basado en nombres de dominio (*vía archivos hosts*) hacia diferentes servicios internos utilizando una única IP externa.

3.4. Estrategia de Almacenamiento Persistente

Para garantizar que los datos de aplicaciones como WordPress no se pierdan, se ha diseñado un sistema de almacenamiento desacoplado:

- **Servidor NFS Dedicado:** Un nodo externo al clúster gestiona los datos físicos en **/srv/nfs/kubedata**.
- **Aprovisionamiento Dinámico:** Se utiliza el **NFS-Subdirectory-External-Provisioner**. Este define una **StorageClass** que automatiza la creación de volúmenes (**PV**) y reclamos (**PVC**), eliminando la necesidad de intervención manual por cada nueva aplicación.

3.5. Automatización y Gestión (IaC)

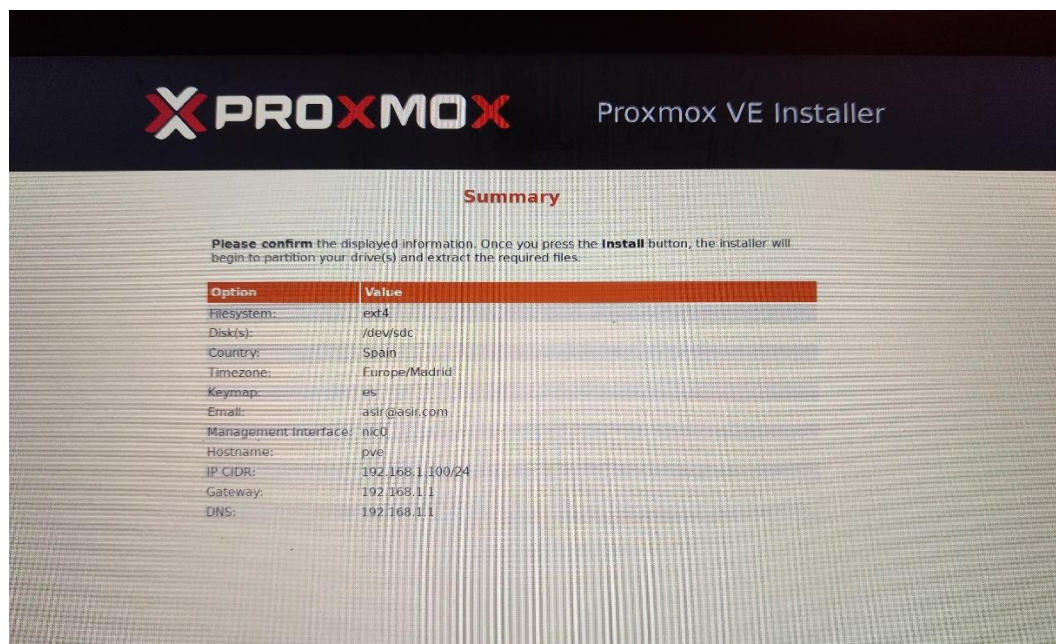
El diseño incluye un servidor de **Ansible** que actúa como orquestador de la configuración. Mediante el uso de archivos de inventario (*hosts.ini*) y *Playbooks*, se asegura que cualquier nuevo nodo (como el **Worker 03**) se configure con los mismos parámetros de red, módulos del kernel (*br_netfilter*) y drivers de *Cgroup* que el resto del clúster.

4. Implementación

La fase de implementación se ejecutó de forma secuencial, partiendo desde el aprovisionamiento del hardware físico hasta la automatización completa del clúster. Cada etapa fue validada antes de proceder a la siguiente para garantizar la estabilidad de la red y el almacenamiento.

4.1. Fase 1: Virtualización y Preparación del Host (Proxmox)

La implementación comenzó con la instalación de **Proxmox VE 9.x** en el servidor físico. Un paso crítico documentado fue el ajuste de la **BIOS**, habilitando el **SVM Mode** para permitir que el procesador gestionara las instrucciones de virtualización de forma nativa, reduciendo drásticamente la sobrecarga de software.

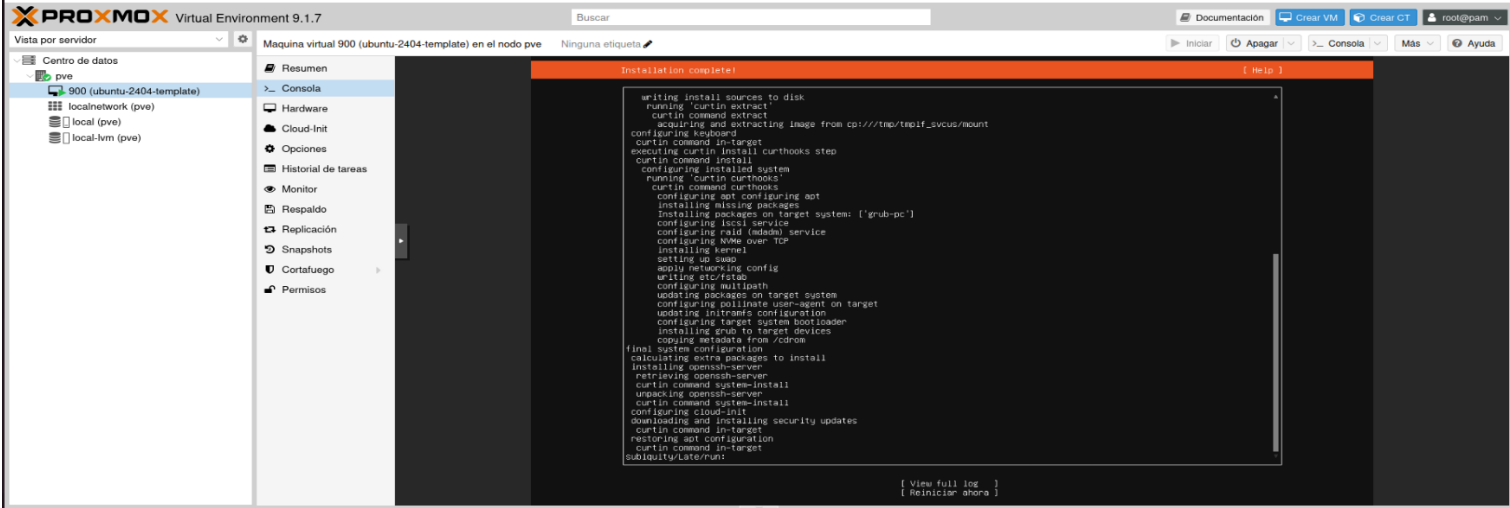


**Programa de instalación de Proxmox VE*

4.1.1. Creación de la Plantilla Base (Gold Template)

Para optimizar el despliegue, se creó una máquina virtual "maestra" con **Ubuntu Server 24.04 LTS**. Esta plantilla incluía:

- **QEMU Guest Agent:** Para permitir la comunicación directa entre *Proxmox* y la VM.
- **OpenSSH Server:** Para la gestión remota.
- **Limpieza de residuos:** Eliminación de archivos temporales para minimizar el peso de los futuros clones.



*Instalación Ubuntu Server en VM desde la web de gestión de Proxmox VE

4.2. Fase 2: Configuración de Red y Nodos

Una vez clonados los nodos (*Master* y *Workers*), se procedió a la configuración de red estática. Un reto técnico resuelto fue la transición en **Netplan** para *Ubuntu 24.04*, donde se descartó el parámetro `gateway4` en favor de rutas explícitas (*routes: to: default*), evitando así pérdidas de conectividad externa tras la asignación de la IP fija.

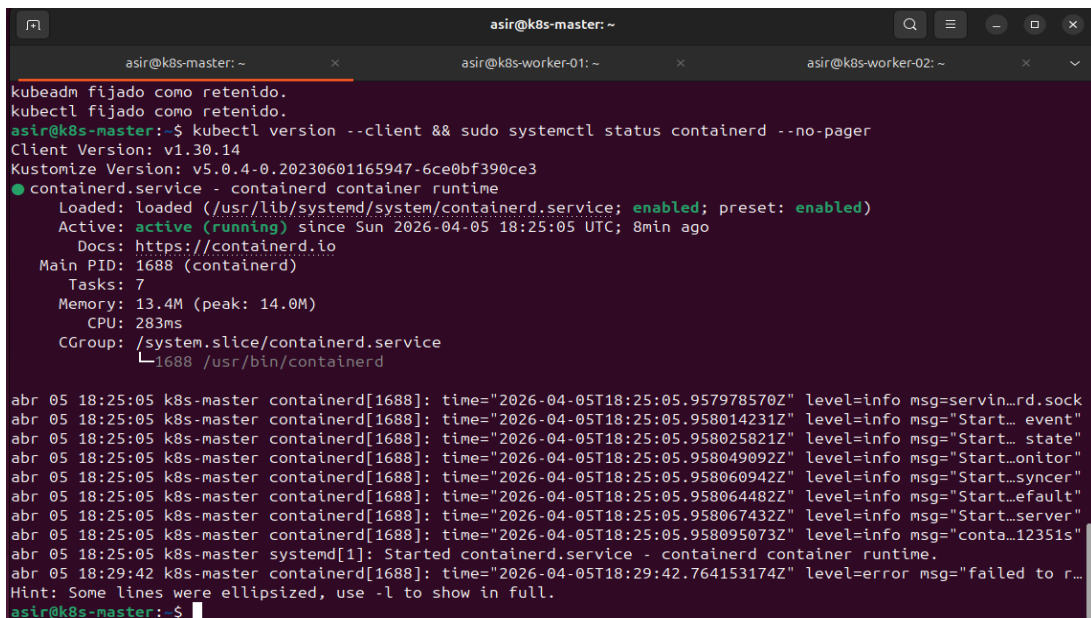
Tipo	Descripción	Uso de dis...	Uso de me...	Uso de CPU	Tiempo de uso	Uso de CP...	Uso de me...
qemu	100 (k8s-master)	0.0 %	8.3 %	48.1% of 2 CPUs	00:00:10	6.0% of 16 ...	2.2 %
qemu	101 (k8s-worker-01)	0.0 %	2.0 %	0.0% of 2 CPUs	00:00:07	0.0% of 16 ...	1.9 %
qemu	102 (k8s-worker-02)	0.0 %	2.8 %	0.0% of 2 CPUs	00:00:04	0.0% of 16 ...	0.4 %
qemu	900 (ubuntu-2404-template)	-	-	-	-	-	-
zone	localnetwork (pve)	-	-	-	-	-	-
storage	local (pve)	12.7 %	-	-	-	-	-
storage	local-lvm (pve)	13.8 %	-	-	-	-	-

*VM clonadas y en ejecución con datos en tiempo real

4.3. Fase 3: Preparación del Kernel y Runtime (*containerd*)

Antes de inicializar *Kubernetes*, se realizaron ajustes profundos en el sistema operativo de cada nodo:

1. **Desactivación de SWAP:** Vital para que el programador de *K8s* gestione la memoria con precisión.
2. **Módulos del Kernel:** Carga manual de *overlay* y *br_netfilter* para permitir el puenteo de red entre pods.
3. **Containerd:** Se instaló como motor de ejecución, configurando explícitamente el parámetro *SystemdCgroup = true* en el archivo *config.toml*. Esto asegura que el proceso *kubelet* y el sistema operativo utilicen el mismo controlador de recursos, evitando inestabilidades bajo carga.



```

asir@k8s-master: ~
kubeadm fijado como retenido.
kubectl fijado como retenido.
asir@k8s-master:~$ kubectl version --client && sudo systemctl status containerd --no-pager
Client Version: v1.30.14
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
● containerd.service - containerd container runtime
   Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-04-05 18:25:05 UTC; 8min ago
     Docs: https://containerd.io
   Main PID: 1688 (containerd)
      Tasks: 7
     Memory: 13.4M (peak: 14.0M)
        CPU: 283ms
    CGroup: /system.slice/containerd.service
            └─1688 /usr/bin/containerd

abr 05 18:25:05 k8s-master containerd[1688]: time="2026-04-05T18:25:05.957978570Z" level=info msg=servin...rd.sock
abr 05 18:25:05 k8s-master containerd[1688]: time="2026-04-05T18:25:05.958014231Z" level=info msg="Start... event"
abr 05 18:25:05 k8s-master containerd[1688]: time="2026-04-05T18:25:05.958025821Z" level=info msg="Start... state"
abr 05 18:25:05 k8s-master containerd[1688]: time="2026-04-05T18:25:05.958049092Z" level=info msg="Start...onitor"
abr 05 18:25:05 k8s-master containerd[1688]: time="2026-04-05T18:25:05.958060942Z" level=info msg="Start...yncer"
abr 05 18:25:05 k8s-master containerd[1688]: time="2026-04-05T18:25:05.958064482Z" level=info msg="Start...efault"
abr 05 18:25:05 k8s-master containerd[1688]: time="2026-04-05T18:25:05.958067432Z" level=info msg="Start...server"
abr 05 18:25:05 k8s-master containerd[1688]: time="2026-04-05T18:25:05.958095073Z" level=info msg="conta...12351s"
abr 05 18:25:05 k8s-master systemd[1]: Started containerd.service - containerd container runtime.
abr 05 18:29:42 k8s-master containerd[1688]: time="2026-04-05T18:29:42.764153174Z" level=error msg="failed to r...
Hint: Some lines were ellipsized, use -l to show in full.
asir@k8s-master:~$
  
```

*Servicio del motor "containerd" activo

4.4. Fase 4: Inicialización y Red de Pods

La creación del clúster se realizó mediante *kubeadm init*. Se definió un CIDR específico para los pods (*10.244.0.0/16*) compatible con *Flannel*, el plugin CNI seleccionado por su ligereza.

Para la unión de los nodos Workers, se aplicó una "Lección Aprendida" crítica: la vinculación manual de la IP interna mediante el archivo */etc/default/kubelet* con la flag *--node-ip*. Este ajuste fue necesario para evitar que *Kubernetes* seleccionara interfaces de red erróneas durante el proceso de arranque.

```

asir@k8s-master: ~
asir@k8s-master: ~
asir@k8s-worker-01: ~
asir@k8s-worker-02: ~
asir@k8s-master:~$ kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
k8s-master    Ready     control-plane  71m   v1.30.14   192.168.1.110 <none>        Ubuntu 24.04.4 LTS   6.8.0-107-generic containerd://1.7.28
k8s-worker-01 Ready     worker      27m   v1.30.14   192.168.1.111 <none>        Ubuntu 24.04.4 LTS   6.8.0-107-generic containerd://1.7.28
k8s-worker-02 Ready     worker      56m   v1.30.14   192.168.1.112 <none>        Ubuntu 24.04.4 LTS   6.8.0-107-generic containerd://1.7.28
asir@k8s-master:~$

```

**Clúster completo en estado Ready listo para su uso*

4.5. Fase 5: Servicios de Red (*MetalLB e Ingress*)

Con el clúster en estado *Ready*, se desplegó la infraestructura de acceso:

- **MetalLB:** Se configuró en modo Capa 2, reservando el rango *192.168.1.200 - 192.168.1.250* de la red física. Se ajustó el *ConfigMap* de *kube-proxy* para activar *strictARP*, requisito indispensable para que *MetalLB* anuncie las IPs correctamente mediante *ARP*.
- **Nginx Ingress Controller:** Se instaló para gestionar el tráfico de Capa 7, permitiendo que servicios como *WordPress* fueran accesibles mediante el dominio *wordpress.asir.local*.

```

asir@k8s-master:~$ kubectl get pods -n metallb-system
NAME                                READY   STATUS    RESTARTS   AGE
controller-86f5578878-28kz9        1/1     Running   0           13m
speaker-4bcsb                      1/1     Running   0           13m
speaker-8n45r                      1/1     Running   0           13m
speaker-jp9px                      1/1     Running   0           13m
asir@k8s-master:~$

```

**Fase 5.1: Pods con el servicio MetalLB en ejecución*

```

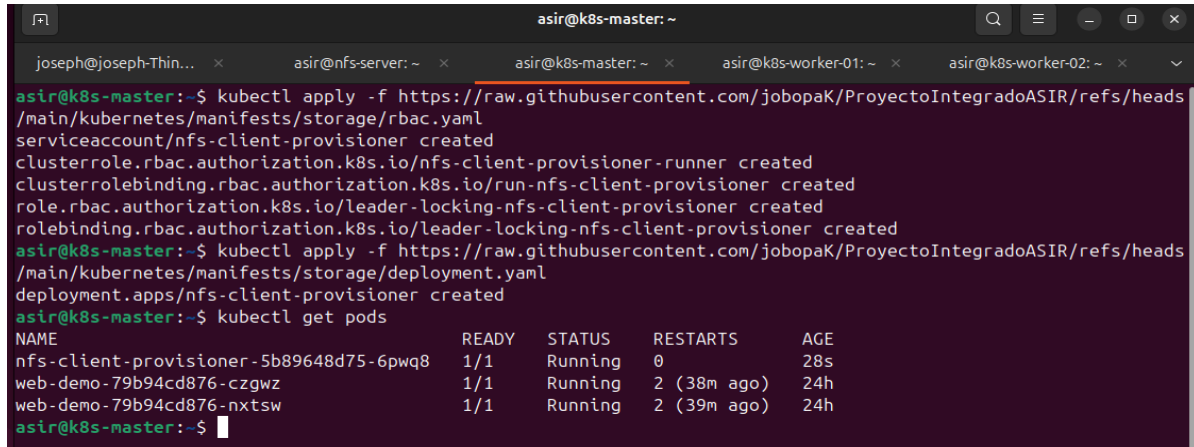
asir@k8s-master: ~
joseph@joseph-ThinkPadX1: ~/ProyectoIntegradoASIR
asir@k8s-master:~$ kubectl get ingress
NAME          CLASS    HOSTS                ADDRESS          PORTS   AGE
web-demo-ingress  nginx    web-proyecto.asir.local  192.168.1.200    80      5m7s
asir@k8s-master:~$ curl -H "Host: web-proyecto.asir.local" http://192.168.1.200
Server address: 10.244.3.7:80
Server name: web-demo-79b94cd876-nxtsw
Date: 13/Apr/2026:07:54:25 +0000
URI: /
Request ID: deb9f2de3c533416e3c7761e64022118
asir@k8s-master:~$

```

**Fase 5.2: Pods con el servicio Ingress en ejecución*

4.6. Fase 6: Almacenamiento Persistente NFS

Se implementó un servidor *NFS* dedicado (IP *192.168.1.116*) con la directiva *no_root_squash*, permitiendo que *Kubernetes* gestionara permisos de archivos de forma remota. La automatización se completó con el **NFS Subdir External Provisioner**, que crea dinámicamente carpetas en el servidor *NFS* para cada aplicación, garantizando que el almacenamiento sea elástico y desacoplado del ciclo de vida del contenedor.



```
asir@k8s-master: ~  
joseph@joseph-Thin... x asir@nfs-server: ~ x asir@k8s-master: ~ x asir@k8s-worker-01: ~ x asir@k8s-worker-02: ~ x  
asir@k8s-master:~$ kubectl apply -f https://raw.githubusercontent.com/jobopaK/ProyectoIntegradoASIR/refs/heads/main/kubernetes/manifests/storage/rbac.yaml  
serviceaccount/nfs-client-provisioner created  
clusterrole.rbac.authorization.k8s.io/nfs-client-provisioner-runner created  
clusterrolebinding.rbac.authorization.k8s.io/run-nfs-client-provisioner created  
role.rbac.authorization.k8s.io/leader-locking-nfs-client-provisioner created  
rolebinding.rbac.authorization.k8s.io/leader-locking-nfs-client-provisioner created  
asir@k8s-master:~$ kubectl apply -f https://raw.githubusercontent.com/jobopaK/ProyectoIntegradoASIR/refs/heads/main/kubernetes/manifests/storage/deployment.yaml  
deployment.apps/nfs-client-provisioner created  
asir@k8s-master:~$ kubectl get pods  
NAME                                READY   STATUS    RESTARTS   AGE  
nfs-client-provisioner-5b89648d75-6pwq8 1/1     Running   0           28s  
web-demo-79b94cd876-czgwz             1/1     Running   2 (38m ago) 24h  
web-demo-79b94cd876-nxtsw             1/1     Running   2 (39m ago) 24h  
asir@k8s-master:~$
```

**Fase 6: Instalación del servicio NFS y pod en ejecución*

4.7. Fase 7: Automatización con Ansible (IaC)

La implementación culminó con la creación del entorno de **Ansible**. Se configuró el acceso SSH mediante llaves y se aplicó la directiva *NOPASSWD* en el archivo *sudoers* de la plantilla base.

- **Playbook de Pre-requisitos:** Automatiza la carga de módulos y desactivación de swap.
- **Super Playbook de Instalación:** Capaz de transformar una VM de *Ubuntu* limpia en un nodo de *Kubernetes* funcional en menos de 5 minutos, incluyendo la gestión dinámica de *tokens* de unión.
- **Seguridad (Ansible Vault):** Se empleó *ansible-vault* para cifrar datos sensibles, como el token de unión al clúster (*token_k8s.txt*) y las credenciales de acceso, asegurando buenas prácticas de seguridad corporativa.

```

asir@ansible-server:~/ansible/playbooks$ ansible-playbook -i ~/ansible/inventory/hosts.ini ~/ansible/playbooks/pre-requisitos.yml
PLAY [Preparación de nodos del clúster] *****
TASK [Gathering Facts] *****
ok: [k8s-worker-01]
ok: [nfs-server]
ok: [k8s-worker-02]
ok: [k8s-master]

TASK [Instalar cliente NFS en todos los nodos] *****
ok: [k8s-worker-01]
ok: [k8s-worker-02]
ok: [nfs-server]
changed: [k8s-master]

TASK [Cargar módulo br_netfilter para el tráfico del clúster] *****
ok: [k8s-worker-01]
ok: [k8s-worker-02]
ok: [k8s-master]
changed: [nfs-server]

TASK [Asegurar que br_netfilter se cargue al reiniciar] *****
changed: [nfs-server]
changed: [k8s-master]
changed: [k8s-worker-02]
changed: [k8s-worker-01]

PLAY RECAP *****
k8s-master      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
k8s-worker-01   : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
k8s-worker-02   : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
nfs-server      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

```

**Fase 7: Ejecución del Playbook realizando las tareas automáticamente*

4.8. Fase 8: Despliegue de Aplicación Real (WordPress)

Con la infraestructura base operativa, se procedió a desplegar una aplicación de arquitectura *Stateful*. Se utilizó *WordPress* respaldado por una base de datos *MySQL*, conectando ambos servicios al servidor *NFS* para garantizar la persistencia de datos. Finalmente, se expuso la web al exterior mediante el *Ingress Controller* y se aplicaron *Kubernetes Secrets* para la gestión segura de contraseñas.

```

asir@k8s-master:~$ kubectl apply -f https://raw.githubusercontent.com/jobopaK/ProyectoIntegradoASIR/refs/heads/main/kubernetes/manifests/apps/wordpress/wordpress.yml
service/wordpress-service created
deployment.apps/wordpress created
asir@k8s-master:~$ kubectl get pods -n wp-proyecto
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-5bb7bd84d8-gcn5b         1/1     Running   0           29s
wordpress-mysql-5794d96555-xhdpt   1/1     Running   1 (5m22s ago)  20h

```

**Fase 8: Lanzamiento del fichero YAML para el despliegue de la aplicación Wordpress*

5. Resultados y pruebas

Esta fase tiene como objetivo validar que la infraestructura cumple con los requisitos de diseño. Se han realizado pruebas de despliegue, estrés y tolerancia a fallos para asegurar que el sistema es apto para un entorno de producción real.

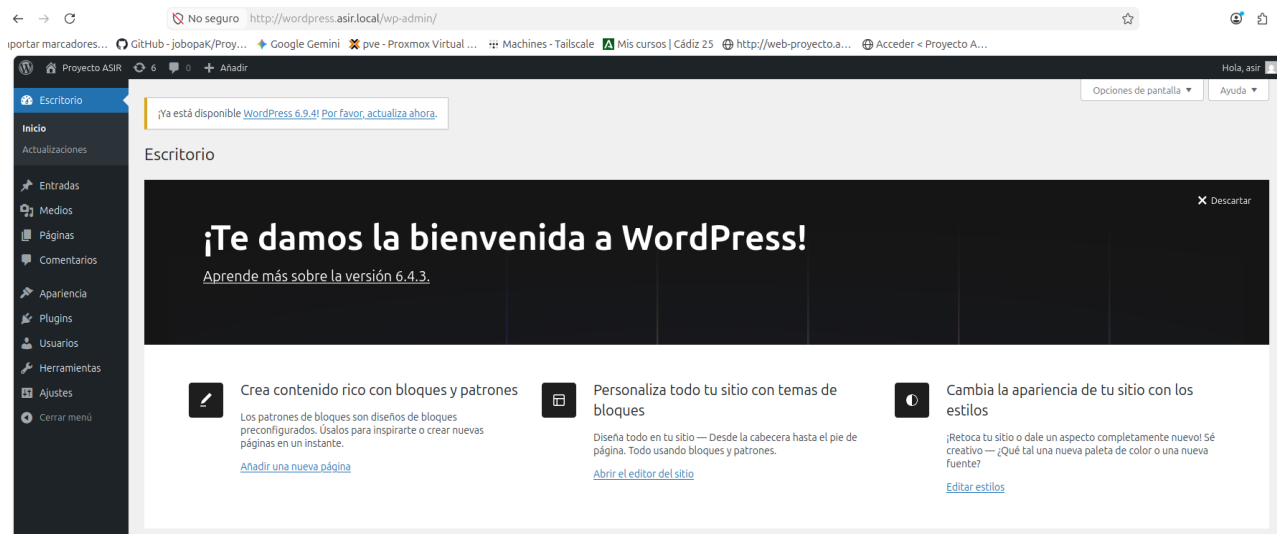
5.1. Despliegue de Servicios Productivos

Se han desplegado dos tipos de cargas de trabajo para testear diferentes protocolos y necesidades de almacenamiento:

5.1.1. Ecosistema WordPress (CMS + DB)

Se desplegó un sitio de *WordPress* utilizando un *Deployment* para la aplicación y otro para la base de datos *MySQL*.

- **Persistencia:** Ambos servicios utilizan *PersistentVolumeClaims (PVC)* vinculados a la *StorageClass* de *NFS*.
- **Acceso:** Se expuso mediante un objeto *Ingress* apuntando al dominio *wordpress.asir.local* en la IP *192.168.1.200*.



*Figura 5.1: Panel de control de Wordpress

5.1.2. Juego Estático: "3 en Raya" (Alta Disponibilidad)

Para probar la ligereza y el balanceo, se desplegó una web estática (*HTML/CSS/JS*).

- **Réplicas:** Se configuraron **2 réplicas** permanentes distribuidas entre los workers.
- **Estrategia:** Se implementó **RollingUpdate**, permitiendo actualizar el código del juego sin interrupción del servicio (*Zero Downtime*).

5.2. Prueba de Alta Disponibilidad (HA)

El test definitivo de resiliencia consistió en la simulación de un fallo crítico de hardware.

1. **Acción:** Se procedió al apagado forzado (*Power Off*) del nodo **k8s-worker-01** desde el panel de *Proxmox* mientras se navegaba en el juego "3 en Raya".
2. **Observación en el Clúster:** El *Master* detectó el estado *NotReady* del nodo. Inmediatamente, el *Ingress Controller* dejó de enviar tráfico a los pods de ese nodo.
3. **Resultado:** El tráfico fue asumido íntegramente por el pod ubicado en **k8s-worker-02**. El usuario final no experimentó pérdida de servicio.

5.3. Prueba de Escalado Dinámico (Horizontal Pod Autoscaling)

Se puso a prueba la capacidad de crecimiento del sistema utilizando el nuevo nodo añadido mediante *Ansible* (**k8s-worker-03**).

- **Comando ejecutado:** *kubectl scale deployment apache-web --replicas=5 -n 3enRaya*.
- **Resultado:** *Kubernetes* distribuyó los 5 pods entre los tres *workers* disponibles. Esta prueba confirmó que la automatización con *Ansible* fue exitosa, ya que el nuevo *worker* aceptó cargas de trabajo de inmediato sin intervención manual adicional.

5.4. Validación de Persistencia y Aprovisionamiento

Se verificó que el **NFS Subdir External Provisioner** gestiona correctamente el almacenamiento físico en el servidor *NFS* (*192.168.1.116*).

1. Se accedió vía *SSH* al servidor *NFS* y se listó el directorio */srv/nfs/kubedata/*
2. Se confirmó la existencia de carpetas con nombres dinámicos (ej. *3enraya-3enraya-pvc-pvc-xxxx*) creadas automáticamente por el clúster.
3. Se realizó una prueba de borrado de Pod: tras eliminar el Pod de base de datos de *WordPress*, el nuevo Pod creado por el *Deployment* se reconectó a la misma carpeta *NFS*, manteniendo todos los artículos y configuraciones previos.

6. Conclusiones

6.1. Cumplimiento de Objetivos

Tras la finalización del proyecto, se confirma el cumplimiento íntegro de los objetivos planteados en la fase de análisis. Se ha logrado desplegar una infraestructura *on-premise* profesional que integra virtualización de alto nivel con orquestación moderna de contenedores. El sistema es capaz de gestionar servicios web de forma balanceada y resiliente, eliminando los puntos únicos de fallo en la capa de aplicaciones.

6.2. Valor Técnico de la Solución

La implementación ha demostrado que es posible construir un entorno de producción de alta fidelidad sin costes de licenciamiento mediante el uso estratégico de software *Open Source*.

- **Resiliencia Real:** La prueba de apagado del *Worker 01* validó que la alta disponibilidad no es solo una configuración teórica, sino una capacidad operativa que garantiza la continuidad del servicio ante fallos de hardware.
- **Soberanía del Almacenamiento:** El uso de un servidor *NFS* dedicado con aprovisionamiento dinámico resuelve el reto de la persistencia de datos en entornos distribuidos, asegurando que la información crítica esté siempre disponible y desacoplada del contenedor.
- **Automatización como Estándar:** La integración de *Ansible (IaC)* permite que la infraestructura sea escalable y replicable. El tiempo de despliegue de un nuevo nodo se ha reducido de horas a escasos minutos, garantizando la consistencia de todas las configuraciones de red y kernel.

6.3. Desafíos Superados y Aprendizajes

El desarrollo de este proyecto ha supuesto un reto técnico significativo, especialmente en la resolución de problemas de red en entornos *bare-metal* virtualizados.

- La depuración de errores en el *CNI Flannel* y la forja de la comunicación IPv4 exclusiva fueron hitos clave para entender el funcionamiento interno de *Kubernetes*.

- La transición a *Ubuntu 24.04* obligó a profundizar en herramientas modernas como *Netplan* y la jerarquía de *Cgroups* de *systemd*, conocimientos altamente demandados en el mercado laboral actual de administración de sistemas.

6.4. Líneas de Trabajo Futuras

Aunque el núcleo de la infraestructura es estable y funcional, el diseño permite una evolución continua:

- **Monitorización Avanzada:** La implementación planificada de *Prometheus* y *Grafana* permitirá una visibilidad total del rendimiento del clúster y la generación de alertas preventivas.
- **Seguridad (Hardening):** Se plantea la integración de políticas de red (*Network Policies*) más restrictivas y la gestión de secretos mediante herramientas externas como *HashiCorp Vault*.
- **Integración Continua (CI/CD):** La infraestructura está lista para conectarse a pipelines de *GitLab* o *Jenkins*, automatizando el ciclo de vida completo desde el código hasta el despliegue en producción.

6.5. Valoración Final

Este proyecto representa la culminación práctica de los conocimientos adquiridos en el ciclo de **ASIR**. No solo demuestra capacidad técnica en redes, seguridad y servidores, sino también una mentalidad orientada a la automatización y la resiliencia. La infraestructura resultante es una plataforma robusta, lista para sostener aplicaciones críticas en un entorno empresarial real, combinando el control del hardware local con la flexibilidad de la nube.

7. Anexos y Referencias

7.1 Webgrafía y Documentación Oficial

A continuación, se detallan las fuentes de información primarias utilizadas para el análisis, diseño e implementación de la infraestructura:

7.1.1. Virtualización y Sistema Base

- [Proxmox VE Administration Guide](#): Documentación oficial para la instalación, configuración de almacenamiento y gestión de redes en el hipervisor.
- [Ubuntu Server Documentation](#): Guías de referencia para la configuración del sistema operativo base de los nodos.

7.1.2. Orquestación y Contenedores (Kubernetes)

- [Kubernetes Documentation](#): Fuente principal para los conceptos de arquitectura (Control Plane, Nodes), despliegue de Pods y Servicios.
- [Containerd Getting Started](#): Referencia para la configuración del runtime de contenedores compatible con CRI.
- [Herramienta Kubeadm](#): Documentación específica para la inicialización del clúster (bootstrapping).
- [Flannel CNI Plugin](#): Repositorio oficial del plugin de red.
- [Troubleshooting Kubeadm](#): Documentación interna sobre resolución de errores en el despliegue de nodos worker.

7.1.3. Redes, Balanceo de Carga y Almacenamiento

- [MetalLB Documentation](#): Guía de implementación para el balanceador de carga en entornos *bare-metal* (capa 2).
- [Ingress Nginx Controller](#): Documentación de la comunidad de Kubernetes para la configuración del controlador de entrada y reglas de enrutamiento.
- [NFS Server Documentation \(Ubuntu\)](#): Referencia técnica para la configuración de exportaciones seguras del servidor NFS.

7.1.4. Automatización y Gestión

- [Ansible Documentation](#): Manuales de referencia para la creación de inventarios, Playbooks y uso de módulos.
- [Proxmox Ansible Collection](#): Documentación de los módulos comunitarios para gestionar Proxmox vía Ansible.

Proyecto Integrado de Grado Superior ASIR

© 2026 - [jobopaK](#)